Prep before starting ROS2:

1. Connect the Arduino to PC using USB.
2. Ensure the fog machine is powered on and wait until is fully heated (you should hear a clicking noise, meaning it is ready to go).
3. Plug in DMX cable into fog machine.
4. Ensure that you have ROS2 Foxy and pyserial installed on Ubuntu 20.04. Also, follow ROS2 tutorials ("Using colcon to build packages") to ensure that the following runs smoothly.

IF USING WINDOWS (WSL):

Package Location (My Computer):

\\wsl.localhost\Ubuntu-20.04\home\fbism\ros2_ws\src\fog_screen_controller

➔ Can be downloaded from GitHub

- `package.xml` file containing meta information about the package
- `resource/<package_name>` marker file for the package
- `setup.cfg` is required when a package has executables, so `ros2 run` can find them
- `setup.py` containing instructions for how to install the package
- `<package_name>` - a directory with the same name as your package, used by ROS 2 tools to find your package, contains `__init__.py`

➔ Custom python node is located in the fog_screen_controller (2nd) folder within the location provided above

Open PowerShell (terminal) and run:

```
wsl -d Ubuntu-20.04
```

Open PowerShell as Administrator:

(Optional) Run: `usbipd list`

➔ See that Arduino is shared

Run: `usbipd attach --busid 1-9 --wsl Ubuntu-20.04`

➔ Will connect USB to WSL2

```
PS C:\Windows\system32> usbipd attach --busid 1-9 --wsl Ubuntu-20.04
usbipd: info: Selecting a specific distribution is no longer required. Please file an issue
ou.
usbipd: info: Using WSL distribution 'Ubuntu-20.04' to attach; the device will be available
usbipd: info: Detected networking mode 'nat'.
usbipd: info: Using IP address 192.168.144.1 to reach the host.
```

Go back to PowerShell (Terminal) and run:

```
cd ~/ros2_ws
colcon build --packages-select fog_screen_controller
source install/local_setup.bash
```

- ➔ First, navigating to the ROS2 workspace will give you access to the ROS2 packages, which are all located in ~/ros2_ws/src. Colcon build will build the package for use. Source your workspace will allow you to use packages and tools in that specific terminal session.

To Start Fog Controller Node:

```
ros2 run fog_screen_controller fog_node
```

- ➔ The command `ros2 run` launches an executable (a ROS2 node) from a package. In this case, the controller package is running the Python node that will inform the Arduino which ROS2 service was called (on or off).

Open second terminal:

```
wsl -d Ubuntu-20.04
source ~/ros2_ws/install/setup.bash
```

To call ROS2 services:

```
ros2 service call <service_name> <service_type> <arguments>
```

- ➔ Both /fog_machine/turn_off and /fog_machine/turn_on have services types 'Empty', so there are no arguments. The Empty type means the service call sends no data when making a request and receives no data when receiving a response.

To turn fog machine on:

```
ros2 service call /fog_machine/turn_on std_srvs/srv/Empty
```

To turn fog machine off:

```
ros2 service call /fog_machine/turn_on std_srvs/srv/Empty
```

```
fbism@BismaLenovo:/mnt/c/Users/fbism$ ros2 service call /fog_machine/turn_o
n std_srvs/srv/Empty
requester: making request: std_srvs.srv.Empty_Request()

response:
std_srvs.srv.Empty_Response()

fbism@BismaLenovo:/mnt/c/Users/fbism$ ros2 service call /fog_machine/turn_o
ff std_srvs/srv/Empty
waiting for service to become available...
requester: making request: std_srvs.srv.Empty_Request()

response:
std_srvs.srv.Empty_Response()

fbism@BismaLenovo:/mnt/c/Users/fbism$ ros2 service call /fog_machine/turn_o
n std_srvs/srv/Empty
waiting for service to become available...
requester: making request: std_srvs.srv.Empty_Request()

response:
std_srvs.srv.Empty_Response()

fbism@BismaLenovo:/mnt/c/Users/fbism$ ros2 service call /fog_machine/turn_o
ff std_srvs/srv/Empty
requester: making request: std_srvs.srv.Empty_Request()

response:
std_srvs.srv.Empty_Response()
```

```
fbism@BismaLenovo:~/ros2_ws$ cd ~/ros2_ws/
fbism@BismaLenovo:~/ros2_ws$ source install/setup.bash
fbism@BismaLenovo:~/ros2_ws$ ros2 run fog_screen_controller fog_node
[INFO] [1755624600.473108408] [fog_device_controller]: Fog machine turned ON
[INFO] [1755624604.886997869] [fog_device_controller]: Fog machine turned OFF
[INFO] [1755624652.913193782] [fog_device_controller]: Fog machine turned ON
[INFO] [1755624656.855997486] [fog_device_controller]: Fog machine turned OFF
```

How does this work? **(Difference between ROS1 version and ROS2 version):**

The ROS1 version controller used rosserial to talk to the Arduino, as the Arduino was a ROS1 node. In the ROS2 version, rosserial was replaced with a custom Python bridge that uses serial communication between the Arduino and the USB. In this case, the Arduino is not a ROS2 node. The ROS2 driver in this case is the Arduino sketch that controls the fog machine and the Python node that connects ROS2 to the Arduino. The Arduino code waits for a 1 or a 0, indicating that the fog machine is to turn on or off respectively. The serial functions available on Arduino ensured that there was communication between the Arduino and PC through a USB. The Python node sends the 1 or 0 when the ROS2 service (turn_on or turn_off) is called.

The code for both ROS1 and ROS2 versions are virtually identical, but the code structure was reorganized to support serial communication over USB in the ROS2 environment.

Arduino Code (Double Click Below):

NewFogDeviceSketch.ino

```
#define RESET_RELAY_PIN 6  // Pin for resetting the Relay (OFF)
#define SET_RELAY_PIN 7    // Pin for setting the Relay (ON)
#define RECTIFIER_INPUT 5  // Pin for the Rectifier
#define GROUND_PIN 9       // Pin for Ground

inline bool isReady() {
  if (digitalRead(RECTIFIER_INPUT) == HIGH) return true;  // If the rectifier is ready, return true
  else return false;
}

void setup() {
  Serial.begin(57600); // starts serial communication
  pinMode(RESET_RELAY_PIN, OUTPUT);  // Set pin 6 as an output
  pinMode(SET_RELAY_PIN, OUTPUT);    // Set pin 7 as an output
  pinMode(RECTIFIER_INPUT, INPUT);   // Set pin 5 as an inputs
  pinMode(GROUND_PIN, OUTPUT);       // Set pin 9 as output

  digitalWrite(GROUND_PIN, LOW);       // The GROUND_PIN is used as ground
  digitalWrite(RESET_RELAY_PIN, LOW);  // Set the PIN to low when it starts
  digitalWrite(SET_RELAY_PIN, LOW);
}

// Turn on the fog machine by setting the relay
void FogTurnOn() {
  digitalWrite(SET_RELAY_PIN, HIGH);
  delay(200);                          // Wait 200 milliseconds for the relay to change state
  digitalWrite(SET_RELAY_PIN, LOW);  // We save energy by turning off the pin
```

NewFogDeviceSketch.ino

```
}

// Turn off the fog machine by resetting the relay
void FogTurnOff() {
  digitalWrite(RESET_RELAY_PIN, HIGH);
  delay(200);
  digitalWrite(RESET_RELAY_PIN, LOW);
}


void loop() {
  if (Serial.available()) { // reports the presence of data in a serial recieve buffer
    char fog = Serial.read();

    if (fog == '1') { // robot wants fog machine on

    //  THE FOLLOWING COMMENTED OUT (NOT NEEDED ATM)
      // bool first_iteration = true;
      // while (!isReady()) {
      //   if (first_iteration) {  // Tell the client that the fog machine is warming up only one time
      //     Serial.println("Fog machine is warming up");
      //     first_iteration = false;
      //   }
      //   delay(10);
      // }

      FogTurnOn();
      Serial.println("Fog machine turned ON");

    } else if (fog == '0') { // robot wants fog machine off
      FogTurnOff();
      Serial.println("Fog machine turned OFF");
    }
  }
}
```

Custom Python Node (fog_node.py):

```
GNU nano 4.8                                fog_node.py                                Modified
from std_srvs.srv import Empty
#communication with Arduino over USB
import serial
#used to dynamically find Arduino's serial port
import glob

# creates ROS2 node called fog_device_controller
class FogDeviceController(Node):
        def __init__(self):
                super().__init__('fog_device_controller')

#finds devices matching /dev/ttyACM*, and [0] means first match, assuming it is Arduino
                port = glob.glob('/dev/ttyACM*')[0]
#opens serial connection at 57600 baud rate (specified in README)
                self.arduino = serial.Serial(port, 57600, timeout=1)

# creates two ROS2 services (will send 1 or 0 to Arduino)
                self.create_service(Empty, '/fog_machine/turn_on', self.turnOnCallback)
                self.create_service(Empty, '/fog_machine/turn_off', self.turnOffCallback)

#sends 1 to Arduino to turn on fog machine
#logs action in terminal the executable (this node) was launched in (where package was built)
        def turnOnCallback(self, request, response):
                self.arduino.write(b'1')
                self.get_logger().info('Fog machine turned ON')
                return response

#sends 0 to Arduino to turn off fog machine
        def turnOffCallback(self, request, response):
                self.arduino.write(b'0')
                self.get_logger().info('Fog machine turned OFF')
                return response

# initializes ROS2 and spins the node to keep it alive, will also clean up at termination
def main(args=None):
        rclpy.init(args=args)
        node = FogDeviceController()
        rclpy.spin(node)
        node.destroy_node()
        rclpy.shutdown()

# wraps logic inside of main() function
if __name__ == '__main__':
        main()
```